Reinforcement Learning Lab

Lesson 11: DDPG, extended environments and reward shaping

Alberto Castellini and Riccardo Fidanza

University of Verona
email: alberto.castellini@univr.it, riccardo.fidanza@studenti.univr.it

Academic Year 2024-25

UNIVERSITÀ di VERONA
Dipartimento di INFORMATICA

Environment Setup (CleanRL) - 1/3

We recommend using **Python 3.10** or **Python 3.11**. Create a virtual environment to isolate the project dependencies.

- **Conda (Recommended)**

```
conda create -n cleanrl python=3.11
conda activate cleanrl
```

- **venv (Unix / Linux / macOS)**

```
python3 -m venv .venv
source .venv/bin/activate
```

- **venv (Windows)**

```
python -m venv .venv
.\.venv\Scripts\activate
```

Note: If PowerShell blocks execution, run `Set-ExecutionPolicy RemoteSigned -Scope CurrentUser` first.

Environment Setup (CleanRL) - 2/3

Once the environment is active, install the necessary dependencies.

- **Method A: Using requirements.txt (file in folder tools)**

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

- **Method B: Manual Installation (Alternative)**
  If *requirements.txt* causes conflicts, you can manually install the core packages needed for CleanRL:

```
python -m pip install --upgrade pip
pip install torch numpy gymnasium matplotlib tensorboard setuptools
```

- **Method C: Manual Installation (conda)**
  (Recommended if you created the environment with Conda)

```
conda install pytorch numpy gymnasium matplotlib tensorboard setuptools -c pytorch
```

Notice: Remember to run these commands *inside* the activated `cleanrl` or `.venv` environment.

Environment Setup (CleanRL) - 3/3

Update the repository of the lab:

```
cd RL-Lab
git stash
git pull
git stash pop
```

Training the RL Agent

You can train the agents using the provided scripts. These will automatically save the models and learning curves in the `runs/` folder.

- **Windows:** `.\lesson11_cleanRL_ddpg_run_train.bat`
- **Unix / macOS:** `chmod +x lesson11_cleanRL_ddpg_run_train.sh` then `./lesson11_cleanRL_ddpg_run_train.sh`

Important Notes for the Lab:

- **Hyperparameters:** Scripts run with default values. You can override them manually from the terminal. Check the argument parsing section in the Python script to see all available options. Example:

```
python lesson11_cleanRL_ddpg_train_code.py --env-id Pendulum-v1 --total-timesteps 5
```

- **Batch Execution:** Inside the `.bat` or `.sh` files, you can modify the code to run a single environment or loop through a list of them.
- **Recommendation:** We strongly advise training **one environment at a time**!

CleanRL: Code Flow

CleanRL is known for its **single-file implementation** approach.

The execution flow is structured as follows:

- **Execution Wrappers (`.bat` or `.sh`):** Scripts used to launch the training process, allowing you to easily run single or multiple environments in a row.
- **Training Script (`lesson11_cleanRL_ddpg_train_code.py`):** A fully self-contained file managing the entire RL pipeline:
  - *parse_args()*: Defines and parses all the hyperparameters.
  - *ReplayBuffer, QNetwork, Actor Class*: Contains the neural network definitions (both Actor and Critic) and the action-sampling logic.
  - *Main Loop*: Sets up vectorized environments, collects rollouts, computes Advantages, and performs the DDPG network updates **(this is where you will implement the missing equations!)**.
- **Testing Script (`lesson11_cleanRL_ddpg_test.py`):** A lightweight script that reconstructs the *Agent*, loads the saved weights (`.cleanrl_model`), and runs a rendered episode to visually evaluate the trained policy.

Today Assignment

In today's lesson, we will implement the update rule of the DDPG algorithm within the *cleanrl* framework and we will run it on some gym environments. On which environments can the DDPG algorithm be used? Check the tables in the next slides.
The file to complete is:

`RL-lab/lessons/lesson11/lesson11_cleanRL_ddpg_train_code.py`

Inside the file, three Python functions are partially implemented. The objective of this lesson is to complete them.

- **def loss_pi_fn**
- **def loss_q_fn**
- **def update_rule**

Your task is to fill in the code where there are #TODO sections.

DDPG to implement (https://spinningup.openai.com/en/latest/algorithms/ddpg.html)

Algorithm 1 Deep Deterministic Policy Gradient

Input: initial policy parameters θ, Q-function parameters ϕ, empty replay buffer $\mathcal{D}$
Set target parameters equal to main parameters $\theta_{\text{targ}} \leftarrow \theta$, $\phi_{\text{targ}} \leftarrow \phi$
repeat
  Observe state s and select action $a = \text{clip}(\mu_\theta(s) + \epsilon, a_{Low}, a_{High})$, where $\epsilon \sim \mathcal{N}$
  Execute a in the environment
  Observe next state s', reward r, and done signal d to indicate whether s' is terminal
  Store (s, a, r, s', d) in replay buffer $\mathcal{D}$
  If s' is terminal, reset environment state.
  if it's time to update **then**
    for however many updates **do**
      Randomly sample a batch of transitions, $B = \{(s, a, r, s', d)\}$ from $\mathcal{D}$
      Compute targets

$$y(r, s', d) = r + \gamma(1 - d)Q_{\phi_{\text{targ}}}(s', \mu_{\phi_{\text{targ}}}(s'))$$

      Update Q-function by one step of gradient descent using

$$\nabla_\phi \frac{1}{|B|} \sum_{(s,a,r,s',d) \in B} \left(Q_\phi(s, a) - y(r, s', d)\right)^2$$

      Update policy by one step of gradient ascent using

$$\nabla_\theta \frac{1}{|B|} \sum_{s \in B} Q_\phi(s, \mu_\theta(s))$$

      Update target networks with

$$\phi_{\text{targ}} \leftarrow \rho\phi_{\text{targ}} + (1 - \rho)\phi$$
$$\theta_{\text{targ}} \leftarrow \rho\theta_{\text{targ}} + (1 - \rho)\theta$$

    end for
  end if
until convergence

DDPG: Introduction

Deep Deterministic Policy Gradient (DDPG) concurrently learns a Q-function and a policy. It uses **off-policy** data and the Bellman equation to learn the Q-function, and uses the Q-function to learn the policy.

- If you know the optimal action-value function $Q^*(s, a)$, then in any given state, the optimal action $a^*(s)$ can be found by solving $a^*(s) = \arg\max_a Q^*(s, a)$.
- **Problem:** When the action space is continuous, the maximization problem is not trivial because we cannot exhaustively evaluate the action space.
- **Solution:** Because the action space is continuous, the function $Q^*(s, a)$ is presumed to be differentiable with respect to the action. DDPG uses an efficient, gradient-based learning rule for a policy $\mu(s)$ which exploits that fact. Then the max is approximated as $\max_a Q(s, a) \approx Q(s, \mu(s))$.

DDPG theory: Q-function loss

Suppose to have a neural network approximator $Q_\phi(s, a)$ of the Q-function, with parameters ϕ, and to have a set $\mathcal{D}$ of transitions (s, a, r, s', d) (where d indicates whether state s' is terminal). Then the loss function for $Q(s, a)$ is:

$$L(\phi, \mathcal{D}) = \underset{(s,a,r,s',d)\sim\mathcal{D}}{\mathrm{E}} \left[\left(Q_\phi(s, a) - \left(r + \gamma(1 - d) \max_{a'} Q_\phi(s', a') \right) \right)^2 \right] \tag{1}$$

But computing the maximum over actions in the target is a challenge in continuous action spaces. **DDPG deals with this by using a target policy network to compute an action which approximately maximizes $Q_{\phi_{\text{targ}}}$.** Then, the final loss for $Q(s, a)$ is:

$$L(\phi, \mathcal{D}) = \underset{(s,a,r,s',d)\sim\mathcal{D}}{\mathrm{E}} \left[\left(Q_\phi(s, a) - \left(r + \gamma(1 - d) Q_{\phi_{\text{targ}}}(s', \mu_{\theta_{\text{targ}}}(s')) \right) \right)^2 \right] \tag{2}$$

DDPG uses DQN's tricks, namely, reply buffer and target networks for Q and μ (updated by polyak averaging, i.e., $\phi_{\text{targ}} \leftarrow \rho\phi_{\text{targ}} + (1 - \rho)\phi$ or $\theta_{\text{targ}} \leftarrow \rho\theta_{\text{targ}} + (1 - \rho)\theta$).

DDPG theory: policy loss

We learn a deterministic policy $\mu_\theta(s)$ which gives the action that maximizes $Q_\phi(s, a)$.

Because the action space is continuous, and we assume the Q-function is differentiable with respect to action, we can just perform gradient ascent (with respect to policy parameters only) to solve

$$\max_\theta \underset{s\sim\mathcal{D}}{\mathrm{E}} \left[Q_\phi(s, \mu_\theta(s))\right]. \tag{3}$$

Note that the Q-function parameters are treated as constants here.

Exploration: To make DDPG policies explore better, we add noise to their actions at training time. The authors of the original DDPG paper recommended time-correlated OU noise, but more recent results suggest that uncorrelated, mean-zero Gaussian noise works perfectly well.

DDPG: Implementation

1. The **CleanRL** implementation is centralized. Unlike other frameworks, networks, buffers, and the update logic coexist in an explicit and straightforward manner.
2. At every valid step, a batch is sampled from the Replay Buffer, and the update is performed in two sequential phases:

```
# 1. Update Q-network (Critic)
qf1_loss.backward()
q_optimizer.step()

# 2. Update Actor-network (Policy)
actor_loss.backward()
pi_optimizer.step()
```

DDPG: Implementation - Q-function Loss

3. The Critic's loss minimizes the error between its prediction and the **Bellman Backup** generated using the target networks.
4. We compute the empirical target $y = r + \gamma(1 - d)Q_{\text{target}}$ by blocking the gradients:

```
with torch.no_grad():
    next_q_value = b_rewards + (1 - b_dones) * gamma * target_q
```

5. We compute the MSE Loss against the empirical target:

```
qf1_loss = F.mse_loss(qf1_a_values, next_q_value)
```

DDPG: Implementation - Policy Loss

6. The Actor must learn a policy $\mu_\theta(s)$ that maximizes the Q-value estimated by the Critic.
7. Since the policy is deterministic, we pass the Actor's actions directly to the Critic to evaluate them: $Q_\phi(s, \mu_\theta(s))$.
8. Because PyTorch optimizers *minimize*, we apply a **negative sign** to perform *gradient ascent*:

```
# Maximize the value estimated by the Critic
actor_loss = -qf1(b_obs, actor(b_obs)).mean()
```

9. At the end of the cycle, we perform a Polyak τ **Soft Update** to gradually update the target network weights without destabilizing the training.

Gymnasium environments

Environment	**States**	**Actions**	Transition
MountainCar-v0	2 Continuous	3 Discrete	Deterministic
CartPole-v1	4 Continuous	2 Discrete	Deterministic
Acrobot-v1	6 Continuous	3 Discrete	Deterministic
LunarLander-v2	8 Continuous	4 Discrete	Deterministic
MountainCarContinuous-v0	2 Continuous	1 Continuous	Deterministic
Pendulum-v0	3 Continuous	1 Continuous	Deterministic
BipedalWalker-v3	24 Continuous	4 Continuous	Deterministic
FrozenLake-v0	16 Discrete	4 Discrete	Stochastic

How can we make DDPG work on discrete state spaces? Can it work on discrete action spaces?

RL Algorithm capabilities

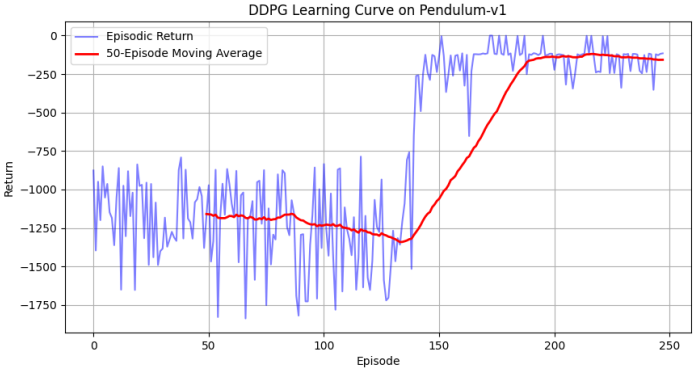
Algorithm	**States**	**Actions**	**Transition**	**On/Off-policy**	**V/Q**	π
Policy gradient	D	D	Needed	Planning	V	No
MC Control	D	D	No	On-policy	Q	No
Sarsa	D	D	No	On-policy	Q	No
Q-Learning	D	D	No	Off-policy	Q	No
Dyna-Q	D	D	Learned	Off-policy	Q	No
DQN	D/C	D	No	Off-policy	Q(2)	No
REINFORCE	D/C	D/C	No	On-policy	No	Yes
REINFORCE+B	D/C	D/C	No	On-policy	V	Yes
A2C	D/C	D/C	No	On-policy	V	Yes
MCTS	D	D	Needed	Planning	Q	No
PPO	D/C	D/C	No	On-policy	V	Yes
DDPG	D/C	C	No	Off-policy	Q	Yes

Expected Results - Pendulum-v1 with DDPG

Once the algorithm is complete it can be run with the command

```
python lesson11_cleanRL_ddpg_train_code.py --env Pendulum-v1 --algo ddpg --exp_name firs
```

It produces textual output in the terminal and a chart with performance over training episodes



Expected Results - Pendulum-v1 with DDPG

When the training concludes, check the newly created experiment folder (e.g., `runs/Pendulum-v1__ddpg__.../`). You will find:

- **Learning Curve (`.png`):** A pre-generated plot showing the episodic return over the training steps (raw and smoothed moving average).
- **Model Weights (`.pth`):** A PyTorch state dictionary containing the trained Actor's weights. To visualize your trained agent playing the game, use the test scripts:

```
.\run_test.bat    # Windows
./run_test.sh     # Unix / macOS
```

Reward shaping: Adapt reward to improve training performance

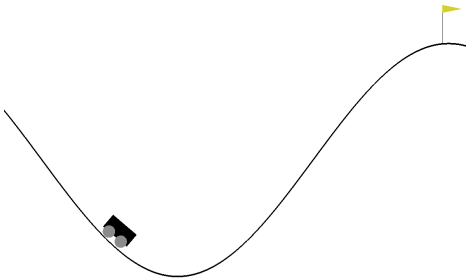
The problem with DDPG performance on MountainCarContinuous-v0 could depend on multiple factors:

- DDPG is very sensible to hyper-parameters and not robust to sparse rewards
- MountainCarContinuous-v0 has sparse reward
- The exploration term could be too small: try to find where it is defined and to change it

The reward function could be rewritten in a **dense form** to help the algorithm finding a good policy. This process is called **Reward Shaping**. To generate a new reward you should first

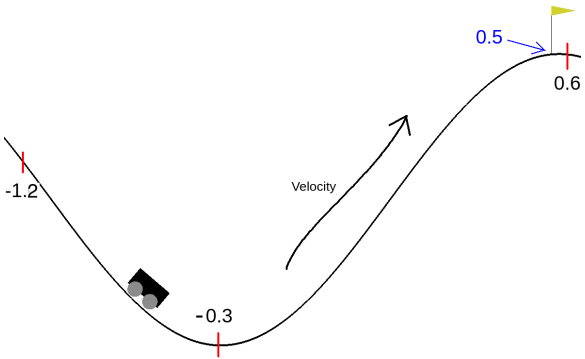
clearly understand the environment.

Environment: MountainCar-v0



- The Mountain Car problem is a classic reinforcement learning problem with the goal of training an agent to reach the top of a hill by controlling a car's acceleration. The problem is challenging because the car is underpowered and cannot reach the goal by simply driving straight up.
- The **state** of the environment is represented as a tuple of 2 values: *Car Position* and *Car Velocity*.
- The continuous **action** represents the directional force applied on the car. The action is clipped in the range [-1,1] and multiplied by a power of 0.0015.
- **Reward:** A negative reward of -0.1 * action2 is received at each timestep. 100 is added when the car reaches the flag.

Understanding the Environment



- The first observation is the position of the car. The values range from -1.2 to 0.6, the central value is -0.3 (see the figure). The task is considered solved if the car reaches position 0.5, i.e., it touches the flag.
- The second observation is the velocity of the car, which assumes positive values if the car is pushing on the right and negative values if it is pushing to the left.
- More information can be found in the step function of the source code here.